

## Лабораторна робота № 2

### Стилізація веб-сторінки засобами CSS

**Тема** Підключення каскадних таблиць стилів і базове оформлення HTML-сторінки наскрізного навчального проекту.

**Мета** Навчитися підключати зовнішній CSS-файл, добирати селектори, застосовувати каскад та успадкування, використовувати модель коробки і CSS-змінні, оформлювати типографіку, навігацію, кнопки та картки контенту.

#### Необхідне програмне забезпечення

Visual Studio Code, Git, обліковий запис GitHub, сучасний браузер із DevTools. Розширення Live Server є рекомендованим, але не обов'язковим.

#### Теоретичні відомості

##### Роль CSS і спосіб підключення

HTML описує структуру та зміст сторінки, а CSS — її візуальне подання. Для навчального проекту стилі слід зберігати окремо від HTML. Зовнішній файл підключають у секції head; атрибут href задає шлях від HTML-файлу до таблиці стилів.

```
<link rel="stylesheet" href="css/styles.css">
```

##### CSS-правило та селектори

CSS-правило складається із селектора та блоку оголошень. Селектор визначає елементи, а кожне оголошення містить властивість і значення. У цій роботі основний спосіб оформлення компонентів — класи: вони повторно використовуються і не залежать від назви HTML-тега.

```
.card-title {  
  color: #1f2937;  
  font-size: 1.25rem;  
}
```

| Селектор   | Приклад       | Призначення                   |
|------------|---------------|-------------------------------|
| елемента   | p             | усі абзаци                    |
| класу      | .card         | усі елементи класу card       |
| вкладений  | .card h3      | заголовки h3 усередині карток |
| псевдоклас | .button:hover | стан під час наведення        |

##### Каскад, специфічність та успадкування

Якщо до елемента застосовуються кілька правил, браузер враховує походження стилю, важливість, специфічність селектора та порядок оголошень. За однакової специфічності перемагає правило, розташоване нижче. Частина властивостей, зокрема color і font-family, успадковується нащадками. Для цієї роботи не використовуйте !important та вбудовані style-атрибути: вони ускладнюють керування каскадом.

## CSS-змінні та базова система оформлення

Спеціальні властивості доцільно оголосити в `:root` і повторно використовувати через `var()`. Назви змінних мають описувати роль значення, а не конкретний відтінок.

```
:root {
  --color-primary: #2563eb;
  --color-text: #1f2937;
  --color-surface: #ffffff;
  --color-background: #f3f4f6;
  --radius: 12px;
  --space: 16px;
}
```

## Модель коробки

Кожен елемент утворює прямокутну коробку: `content`, `padding`, `border` і `margin`. Значення `box-sizing: border-box` робить обчислення розміру передбачуваним, оскільки задана ширина вже містить `padding` і `border`.

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

## Мінімальні базові стилі

Повний `reset.css` у цій роботі не потрібний. Достатньо невеликої нормалізації, яка не видаляє важливі браузерні ознаки доступності. Зокрема, не слід прибирати `outline` для елементів у фокусі.

```
body {
  margin: 0;
  font-family: Inter, Arial, sans-serif;
  line-height: 1.5;
  color: var(--color-text);
  background: var(--color-background);
}

img {
  display: block;
  max-width: 100%;
  height: auto;
}

button, input, select, textarea {
  font: inherit;
}
```

## Стани інтерактивних елементів

Посилання й кнопки повинні мати помітні стани `hover` та `focus-visible`. Стан фокусу важливий для користувачів клавіатури. Контраст тексту і фону має залишатися достатнім для читання.

```

.button:hover {
  background-color: #1d4ed8;
}

.button:focus-visible {
  outline: 3px solid #93c5fd;
  outline-offset: 3px;
}

```

### Завдання

Продовжити вебсайт, створений у лабораторній роботі №1. Індивідуальним варіантом залишається обрана раніше тематика. Потрібно оформити наявну HTML-сторінку, не змінюючи її предметну область і не додаючи JavaScript.

### Обов'язкові вимоги

1. створити й підключити `css/styles.css`;
2. додати зрозумілі класи до основних частин сторінки;
3. визначити щонайменше 5 CSS-змінних: основний колір, колір тексту, фон сторінки, фон поверхні та базовий відступ або радіус;
4. оформити `body`, заголовки, абзаци, навігацію, посилання, одну кнопку та щонайменше 3 однотипні картки;
5. застосувати `margin`, `padding`, `border`, `border-radius` і `box-sizing`;
6. реалізувати `hover` і `focus-visible` для посилань або кнопок;
7. перевірити сторінку в DevTools та виконати перевірку CSS;
8. оформити звіт у файлі `README.md` у корені репозиторію;
9. створити щонайменше 3 змістовні коміти та pull request із гілки `lab-2-css`.

### Адаптація завдання до тематики

| Тематика сайту        | Приклад набору карток                    |
|-----------------------|------------------------------------------|
| Кав'ярня або ресторан | напої, страви або спеціальні пропозиції  |
| Книги або кіно        | книги, автори, фільми або жанри          |
| Подорожі або музей    | місця, експонати або події               |
| Спорт або освіта      | тренування, курси або навчальні події    |
| Інша тема             | три однорідні об'єкти предметної області |

### Хід роботи

- 1. Підготуйте репозиторій.** Перейдіть до репозиторію лабораторної №1, переконайтеся, що основна гілка містить завершений HTML, отримайте актуальні зміни та створіть гілку `lab-2-css`.
- 2. Упорядкуйте структуру.** Створіть папку `css` і файл `styles.css`. Зображення, якщо вони використовуються, зберігайте у папці `images`. Не перейменовуйте `index.html` без потреби.

- 3. Додайте класи.** Перегляньте HTML і додайте класи до header, nav, основних секцій, кнопки, набору карток та footer. Використовуйте короткі змістовні назви: site-header, main-nav, card, card-title, button.
- 4. Підключіть таблицю стилів.** Додайте link у head після метаданих. Відкрийте сторінку через Live Server або безпосередньо в браузері й переконайтеся, що тестове правило застосовується.
- 5. Створіть базові правила.** Додайте box-sizing, базові стилі body, гнучкі зображення та успадкування шрифту для елементів форми. Не вимикайте outline.
- 6. Сформууйте палітру.** Оголосіть CSS-змінні в :root і застосуйте їх щонайменше у трьох різних правилах. Доберіть 2–3 основні кольори, нейтральний фон і читабельний колір тексту.
- 7. Оформіть типографіку.** Встановіть font-family, line-height, розміри заголовків і колір другорядного тексту. Дотримуйтеся не більш як двох гарнітур шрифту.
- 8. Оформіть навігацію та дії.** Стилізуйте посилання й кнопку. Передбачте звичайний стан, hover та focus-visible. Текст посилання повинен залишатися зрозумілим без опори лише на колір.
- 9. Оформіть картки.** Для спільного класу card задайте фон, внутрішній відступ, межу або легку тінь і заокруглення. Усі картки мають мати однакову візуальну систему.
- 10. Перевірте результат.** У DevTools перевірте, які правила застосовані, успадковані або перекриті. Перегляньте сторінку щонайменше у двох сучасних браузерах або у двох режимах емуляції.
- 11. Фіксуйте етапи роботи.** Створіть щонайменше три змістовні коміти. Один коміт має містити один логічно завершений набір змін. Використовуйте формат type: description, наприклад style: add base page styles, style: design cards and buttons, docs: update lab report in README.
- 12. Оформіть README.** У корені репозиторію створіть або оновіть README.md. Додайте тему, мету, опис проєкту, перелік реалізованих змін, структуру файлів, знімки екрана, посилання на pull request і висновок.
- 13. Завершіть роботу з Git.** Перевірте git status і журнал git log --oneline. Переконайтеся, що повідомлення комітів конкретно описують зміни. Відправте гілку на GitHub і відкрийте pull request до основної гілки.

### Критерії готовності

1. сторінка відкривається без помилок, а styles.css коректно підключено;
2. оформлення відповідає тематиці, а текст легко читати;
3. однотипні елементи мають спільні класи й узгоджений вигляд;
4. у CSS немає !important, вбудованих style-атрибутів і невикористаних правил;
5. фокус клавіатури помітний, зображення не виходять за межі контейнера;
6. README.md містить повний звіт, а історія — щонайменше 3 змістовні коміти без повідомлень update, changes або final;
7. pull request містить лише зміни лабораторної роботи №2.

### Питання для самоперевірки

1. Які переваги має зовнішній CSS-файл порівняно з атрибутом style?
2. З яких частин складається CSS-правило?
3. Чим селектор класу відрізняється від селектора HTML-елемента?

4. Як браузер обирає правило, якщо кілька правил задають ту саму властивість?
5. Які властивості можуть успадковуватися від батьківського елемента?
6. Для чого використовують box-sizing: border-box?
7. Чим відрізняються margin і padding?
8. Як оголосити та використати CSS-змінну?
9. Чому не слід без заміни видаляти outline для елементів у фокусі?
10. Яким вимогам має відповідати змістовне повідомлення коміту?

### Звіт у файлі README

Окремий звіт у форматі DOCX не подається. Результати виконання лабораторної роботи оформлюються у файлі README.md у корені репозиторію. README має містити:

1. назву лабораторної роботи, тему, мету та короткий опис проєкту;
2. перелік реалізованих змін та основні використані CSS-властивості й змінні;
3. структуру файлів проєкту після виконання роботи;
4. 1–2 знімки екрана готової сторінки з відносними шляхами до файлів зображень;
5. посилання на pull request і висновок про отримані навички.

### Приклад структури README.md

```
# Лабораторна робота №2
## Тема, мета та опис проєкту
## Реалізовані зміни
## Структура проєкту
## Результат
## Посилання та висновок
```

### Вимоги до повідомлень комітів

Використовуйте формат type: description і конкретно називайте зміну. Допечні типи: style, feat, fix, refactor, docs. Не використовуйте повідомлення update, changes, lab 2 або final.

### Список джерел

1. [MDN Web Docs. Getting started with CSS](#)
2. [MDN Web Docs. CSS selectors](#)
3. [MDN Web Docs. Introduction to the CSS cascade](#)
4. [MDN Web Docs. The box model](#)
5. [MDN Web Docs. Using CSS custom properties](#)
6. [Visual Studio Code. Emmet in Visual Studio Code](#)
7. [W3C. CSS Validation Service](#)
8. [GitHub Docs. Managing branches within your repository](#)
9. [GitHub Docs. About the repository README file](#)
10. [Conventional Commits. Specification 1.0.0](#)